HelixCode Constitution

HelixCode Project Constitution

Version: 1.0.0 **Effective Date**: 2026-04-30 **Scope**: This Constitution applies to HelixCode and ALL its submodules **Authority**: Cascaded from HelixAgent root governance with HelixCode-specific addenda

---

Preamble

HelixCode is an enterprise-grade distributed AI development platform. This Constitution establishes the non-negotiable rules that govern all development, testing, deployment, and maintenance activities within the project. Every contributor, agent, and automated process MUST adhere to these rules. No exceptions.

CONST-001: No CI/CD Pipelines (Permanent)

No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks. All builds and tests run manually or via Makefile/script targets.

Rationale: Manual execution ensures human oversight and prevents automated propagation of bluffs.

CONST-002: No Mocks in Production (Permanent)

CONST-002a: Production Code

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations.

CONST-002b: Test Code

Mocks/stubs/fakes MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`).

Rationale: Production bluffs have repeatedly been discovered where features appeared implemented but were non-functional.

CONST-003: No HTTPS for Git (Permanent)

SSH URLs only (`git@github.com:…`, `git@gitlab.com:…`, etc.) for clones, fetches, pushes, and submodule updates. SSH keys are configured on every service.

CONST-004: No Manual Container Commands (Permanent)

Container orchestration is owned by the project's binary/orchestrator (e.g., `make build` → `./bin/<app>`). Direct `docker`/`podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows.

CONST-005: 100% Real Data for Non-Unit Tests

Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.

Verification: Every integration/E2E test MUST connect to real services or skip (not fail) if unavailable.

CONST-006: Challenge Coverage (Permanent)

Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success — validate actual behavior, not return codes.

CONST-007: Health & Observability

Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.

CONST-008: Documentation & Quality

Update `CLAUDE.md`, `AGENTS.md`, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.

CONST-009: Validation Before Release

Pass the project's full validation suite (`make ci-validate-all`-equivalent) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).

CONST-010: Comprehensive Verification

Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives. Grep-only validation is NEVER sufficient.

CONST-011: Resource Limits for Tests & Challenges

ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1` for `go test`. Container limits required.

CONST-012: Bugfix Documentation

All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` with root cause analysis, affected files, fix description, and a link to the verification test/challenge.

CONST-013: Real Infrastructure for All Non-Unit Tests

Mocks/fakes/stubs/placeholders MAY be used ONLY in unit tests. ALL other test types — integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification — MUST execute against REAL running systems with REAL containers, REAL databases, REAL services, and REAL HTTP calls.

CONST-014: Reproduction-Before-Fix (Mandatory)

Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence:

1. Write the Challenge first
2. Run it; confirm fail (it reproduces the bug)
3. Then write the fix
4. Re-run; confirm pass
5. Commit Challenge + fix together

The Challenge becomes the regression guard for that bug forever.

CONST-015: Concurrent-Safe Containers

Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use thread-safe primitives. Bare `sync.Mutex + map/slice` combinations are prohibited for new code.

CONST-016: Definition of Done (Universal)

A change is NOT done because code compiles and tests pass. "Done" requires pasted terminal output from a real run.

- **No self-certification**: Words like *verified, tested, working, complete, fixed, passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
- **Demo before code**: Every task begins by writing the runnable acceptance demo
- **Real system, every time**: Demos run against real artifacts
- **Skips are loud**: `t.Skip` without a trailing `SKIP-OK: #<ticket>` comment breaks validation

---

CONST-017: Zero-Bluff Testing (CONST-035 Implementation)

A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user**.

Every PASS result MUST guarantee:

- **Quality** - Feature behaves correctly under real user inputs
- **Completion** - Feature is wired end-to-end with no stub gaps
- **Full usability** - A user following documentation succeeds

Bluff Taxonomy (forbidden patterns):

- **Wrapper bluff** - Assertions PASS but wrapper's exit-code logic is buggy
- **Contract bluff** - System advertises capability but rejects it in dispatch
- **Structural bluff** - File exists but doesn't contain working code
- **Comment bluff** - Comment promises behavior code doesn't have
- **Skip bluff** - `t.Skip("not running yet")` without `SKIP-OK` marker

CONST-018: Host Power Management Hard Ban

You may NOT generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition.

Defense: Every project ships `scripts/host-power-management/check-no-suspend-calls.sh` and `challenges/scripts/no_suspend_calls_challenge.sh`.

CONST-019: Container Up ≠ Healthy

Container `Up` status does NOT mean the application is healthy. Application-layer probes are mandatory for every service:

- PostgreSQL: `SELECT 1`
- Redis: `PING`
- LLM Providers: Real generation request
- HTTP Services: `GET /health` with deep checks

CONST-020: Provider Fallback Chain Reality

Every LLM provider fallback chain MUST be tested with actual failures. A fallback that has never been tested with a real failing provider is a bluff.

CONST-021: No Mocks Above Unit Build Target

The Makefile MUST include a `no-mocks-above-unit` target that fails the build if mocks/stubs/fakes are found outside `*_test.go` files.

CONST-022: Submodule Governance Propagation

Every submodule MUST either:

1. Have its own Constitution.md, CLAUDE.md, and AGENTS.md, OR
2. Have a symlink to the parent repository's governance files, OR
3. Have a reference comment in its README pointing to parent governance

No submodule is exempt from these rules.

CONST-023: Docker Health Checks Mandatory

Every Dockerfile MUST include:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8080/health || exit 1
```

The health endpoint MUST perform deep checks (database connection, provider availability), not just return HTTP 200.

CONST-024: Version Pinning

All dependencies MUST be pinned to specific versions in `go.mod`. No `latest`, no floating tags. Renovate or Dependabot (manual review only — see CONST-001) may propose updates.

CONST-025: Secret Management

NO secrets in code. EVER. Secrets via:

- Environment variables (production)
- `.env` files (development, in `.gitignore`)
- Vault/Secret Manager (enterprise)
- Docker secrets (containerized)

`go mod tidy` MUST NOT add secret-scanning bypasses.

CONST-026: Minimal Privilege Containers

Containers run as non-root. Every Dockerfile:

```
RUN adduser -D -u 1001 helixcode
USER helixcode
```

CONST-027: Network Isolation

Container orchestration MUST use internal networks. Services communicate via named hosts, not exposed ports where possible.

CONST-028: Backup Before Destructive Operations

Every file editing tool MUST create backups before modification. The backup MUST be restorable.

CONST-029: Input Validation at All Boundaries

Every public function MUST validate inputs. No trust of caller-provided data. SQL injection, path traversal, command injection MUST be impossible by design.

CONST-030: Graceful Degradation

When external services are unavailable, the system MUST degrade gracefully:

- Return partial results where possible
- Queue operations for retry
- Inform user of degraded state
- NEVER crash or hang indefinitely

CONST-031: Audit Trail

Every significant operation MUST be logged with:

- Timestamp
- User identity
- Operation type
- Success/failure status
- Resource affected

Log retention: 90 days minimum.

CONST-032: Emergency Stop

Every long-running or distributed operation MUST support cancellation via `context.Context`. Users MUST be able to interrupt any operation.

CONST-033: Data Integrity

Database writes MUST be transactional. Partial writes MUST be rolled back. Consistency checks MUST run periodically.

CONST-034: API Stability

Public APIs maintain backward compatibility within major versions. Deprecation requires:

- 6-month notice
- Migration guide
- Compatibility shim

CONST-035: End-User Usability Mandate (2026-04-29 Strengthening)

A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user of the product**.

The HelixAgent project has repeatedly hit the failure mode where every test ran green AND every Challenge reported PASS, yet most product features did not actually work. This MUST NOT recur in HelixCode.

Every PASS result MUST guarantee: a. **Quality** - correct behavior under real inputs, edge cases, concurrency b. **Completion** - wired end-to-end with no stub/placeholder gaps c. **Full usability** - a user following documented request shapes SUCCEEDS

A passing test that doesn't certify all three is a **bluff** and MUST be tightened.

Bluff taxonomy (each pattern observed and now forbidden):

- **Wrapper bluff** - assertions PASS but wrapper's exit-code logic is buggy
- **Contract bluff** - system advertises capability but rejects it in dispatch
- **Structural bluff** - `check_file_exists` passes but doesn't run the test
- **Comment bluff** - comment promises behavior code doesn't actually have
- **Skip bluff** - `t.Skip("not running yet")` without `SKIP-OK: #<ticket>` marker

Full background: `docs/HOST_POWER_MANAGEMENT.md` and this Constitution (CONST-035).

CONST-036: Propagation to Submodules

This Constitution, along with CLAUDE.md and AGENTS.md, MUST be propagated to ALL submodules. Each submodule's governance MUST reference this parent Constitution. Changes to this Constitution MUST trigger review of all submodule governance files.

Amendment Process

Constitution amendments require:

1. Written proposal with rationale
2. Challenge demonstrating the need
3. 72-hour review period
4. Approval by project architect
5. Update to all submodule governance files

This Constitution is the supreme law of the HelixCode project. No code, test, or process may contradict it.